CSS Typography, Box Model & Layout — and Our First Steps into JavaScript

Introduction to Web Technology — Week 7

Index

- 1. Where We Left Off: Settling CSS Conflicts — *Who wins when rules fight? (Spoiler: the cascade referees.)*
- 2. Styling Fonts and Text — *Pick a font, but always pack a backup.*
- 3. The Box Model: Every Element Is a Box — *Everything on a web page is secretly a cardboard box.*
- 4. Background — *Wallpaper for your boxes.*
- 5. Borders — *The picture frame, now with rounded corners and drop shadows.*
- 6. Controlling Layout: Flow and Positioning — *Teaching boxes where to sit.*
- 7. CSS Layouts: Columns, Floats and Position Values — *Text wrapping around images, the old-school way.*
- 8. Flexbox vs Grid — *Stretchy rubber bands versus a rigid spreadsheet.*
- 9. Flexbox in Detail — *One axis, many tricks.*
- 10. Flex and Grid Together — *Best friends: Grid builds the house, Flex arranges the furniture.*
- 11. Responsive Design — *One page, every screen, no squinting.*
- 12. Module Recap: HTML and CSS — *The greatest hits album.*
- 13. Module 3 Begins: Client-Side Programming with JavaScript — *Making pages react instead of just sitting there.*
- 14. Where JavaScript Is Used — *Spoiler: practically everywhere.*
- 15. Client-Side Scripting Architecture — *Who does what, and where.*
- 16. Why Client-Side? Pros and Cons — *Every superpower has a price tag.*
- 17. Contemporary JavaScript Coding — *The modern request dance, step by step.*
- 18. Where JavaScript Lives in a Page — *Inline, embedded or external: choose wisely.*
- 19. Declaring Variables — *let, const, and the var you should leave behind.*
- 20. Practice Questions — *Think before you scroll to the answers (there are none).*

1. Where We Left Off: Settling CSS Conflicts

CSS rules can live in several places: **external** style sheets, **embedded** style blocks, and **inline** styles. When several rules apply to the same element, the browser needs a way to decide which one wins. The decision depends on:

- whether the rule is external, embedded or inline,
- its position in the document tree (hierarchy),
- proximity to the element,
- whether the relation is sibling or descendant.

This mechanism is built from **specificity** and **inheritance**, and together with source order it forms the **cascade**. With conflict resolution handled, we move on to the other aspects of CSS. Since CSS exists mainly for **presentation**, we start with fonts and text.

2. Styling Fonts and Text

How text is presented matters a great deal. CSS gives us a family of properties for it:

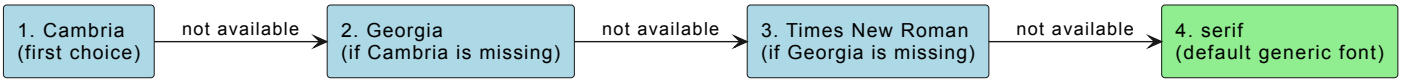
- `font-family` takes a **list of fonts in order of preference**, and should always end in a **generic fallback** like `sans-serif`.
- `font-size`, `font-weight`, `font-style` control **size, boldness and italics** respectively.
- `color` sets the **text color** — easy to confuse with `background-color`, which is the **box behind it**.
- `text-align`, `line-height`, `letter-spacing` are the **finer controls** for spacing and alignment.

You are encouraged to try these variations in your CSS assignments, designing your front end with fonts that suit its purpose.

How the Fallback List Works

```
p { font-family: Cambria, Georgia, "Times New Roman", serif; }
```

Here `p` is the HTML element for a **paragraph**; any text inside it receives this font family.



Why a fallback list? The actual rendering depends on the **browser and system** where the page is displayed. If your first-choice font is not installed, the browser tries the second, and so on. A generic family (`serif`, `sans-serif`, …) is supported by essentially every browser environment, so it is the safety net.

> **Note for Review** Pro-Tip: Exams love the question "What happens if none of the listed fonts is installed?" Remember that the browser falls back to the **generic family** named at the end of the list. Always end the list with one, and note that a font name containing spaces (like `"Times New Roman"`) goes in quotes.

The Different Generic Font Families

Generic Name	Description
`serif`	Letters carry small decorative strokes ("serifs") at the ends of the glyphs
`sans-serif`	"Without serif" — clean glyph ends
`monospace`	Every letter has the **same width**
`cursive`	Handwriting-like decorative style
`fantasy`	Decorative style

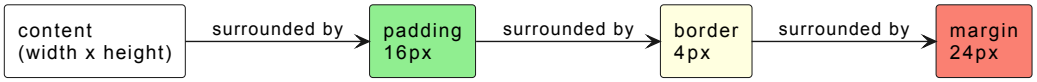
- In a **monospace** font each letter occupies the same width (think of `T h i s` in equal cells). In a regular **proportionally-spaced** font each letter has a **variable width** (the `i` is narrower than the `T`).
- **Decorative and cursive fonts vary from system to system and are therefore rarely used.**

3. The Box Model: Every Element Is a Box

The box model is a critical part of HTML + CSS. Every element you render — text, media, tables, images — is presented as a **box**. An HTML template is really a set of containers: boxes of various sizes with different **padding, border and margin**.

A box has four layers, from inside to outside:

1. **Content** — the actual width × height holding the text/image.
2. **Padding** — space between content and border.
3. **Border** — the line around the padding.
4. **Margin** — the transparent space separating this box from others.



The total space an element reserves is the sum of all these layers:

$$\text{total element size} = \text{content} + \text{padding} + \text{border} + \text{margin}$$

Applied along one dimension (both sides count), with the slide's values, padding $= 16$px, border $= 4$px, margin $= 24$px:

$$\text{total width} = W_{content} + 2(16) + 2(4) + 2(24) = W_{content} + 88 \text{ px}$$

> **Note for Review** Pro-Tip: Padding is *inside* the border (it shows the element's background), margin is *outside* (always transparent). If a question asks "how much space does this element occupy on the page?", add **all four** layers on **both** sides.

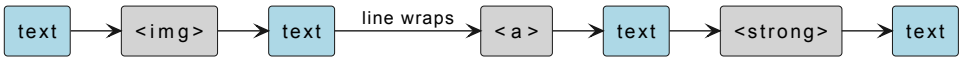
The box model lets you customize sizes and manage space. If you open any website in the browser **developer tools**, you will see a **box-model indicator** (at the lower right) showing exactly how that page's elements are organized.

Inline Elements

Inline content is **laid out horizontally, left to right, within its container**. Once a line is filled, the **next line receives the remaining content**, and so on — a continuous flow across multiple lines.

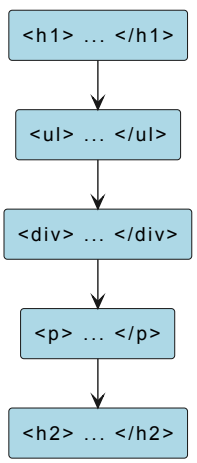
```
<p>
This photo <img src="photo-con.png"> of Conservatory Pond in
<a href="http://www.centralpark.com">Central Park</a> was
taken with a <strong>Canon EOS 30D</strong> camera.
</p>
```

Here the `<p>` is the container (opening tag to closing tag). The text, `<img>`, `<a>` and `<strong>` are all inline items placed in sequence; the content of this `<p>` ends up displayed on **two lines**.



Block-Level Elements

A block-level element does **not** flow inline; **each block sits on its own line** and is displayed in **normal flow from the top of the browser window to the bottom**.



- By default each block-level element **fills the entire width of its parent** (here the parent is `<body>`, which equals the width of the browser window). So the sibling boxes are homogeneous in width.
- You can use CSS box-model properties to customize, for instance, the **width of the box** and the **margin space** between block-level elements.

Parent–child vs sibling. In this example the `h1`, `ul` (an unordered list, i.e. a list without numbering), `div`, `p` and `h2` all sit at the **same level**; they are **siblings**, so all receive the full width of their common parent. A true parent would be something like `body` containing them (or `html` containing `head` and `body`). The container's width decides every sibling's width.

4. Background

Just as space organization matters, so does the background. Using CSS you can place an image as the background and control how it behaves, much like inserting images into a PowerPoint slide or a document.

Goal	CSS
Single image at top-left corner, not repeated	`background-image: url(checkers.png);` `background-repeat: no-repeat;`
Place the image at an offset (300px from left, 50px from top)	`background-image: url(checkers.png) no-repeat;` `background-position: 300px 50px;`
Tile the image across the entire area	`background-repeat: repeat;`
Tile only vertically	`background-repeat: repeat-y;`
Tile only horizontally	`background-repeat: repeat-x;`
Scale image to **cover** the whole box (may crop)	`background-size: cover;`
Scale image to **fit entirely inside** the box	`background-size: contain;`
Explicit size	`background-size: 200px 100px;`

Choosing the area the background is drawn in with `background-origin`:

```
/* background starts at the outer edge of the border */
background-origin: border-box;
background-size: cover;
border-size: 5px;
border-color: rgba(242,112,90,0.5);

/* background starts inside the padding */
background-origin: padding-box;

/* background starts inside the content area */
background-origin: content-box;
padding: 10px;
```

> **Note for Review** Pro-Tip: `background-position` and `repeat` work together. `no-repeat` + `300px 50px` places **one** copy 300px from the left and 50px from the top of the frame of reference. Also, the slide writes `border-size`; the standard property name is `border-width`.

5. Borders

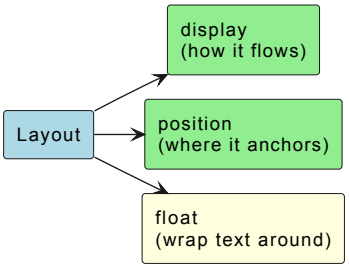
Borders can be dotted, straight, dashed, curved at corners, and so on.

Effect	CSS
Solid line	`border-style: solid;`
Dotted line	`border-style: dotted;`
Dashed line	`border-style: dashed;`
No border	`border-width: 0;`
Thin border	`border-width: 1px;`
Different width per side (top right bottom left)	`border-width: 1px 2px 0 1px;`
Shorthand: style, width, color	`border: solid 2px gold;`
Rounded corners with a border	`border-radius: 3px;` `border-width: 1px;`
Rounded corners, no border	`border-radius: 3px;` `border-width: 0;`
Shadow offset right/down	`box-shadow: 2px 2px gray;`
Blurred shadow around all sides	`box-shadow: 2px 2px 2px 2px gray;`

> **Note for Review** Pro-Tip: For multi-value `border-width`, remember **T-R-B-L**: clockwise starting from the top. So `1px 2px 0 1px` means top 1, right 2, bottom 0, left 1.

6. Controlling Layout: Flow and Positioning

Layout is about **flow** and **positioning**.



- In **normal flow**, **block elements stack vertically** (one under the other) and **inline elements flow left to right**.
- `display` — switches an element's flow behaviour: `block`, `inline`, `inline-block`, `flex`, `grid`, `none`. It decides *how the element flows on the page*.
- `position` — **static by default**; `relative`, `absolute`, `fixed` and `sticky` each anchor the element differently.
- `float` **predates Flexbox and Grid** — you will still run into it for **wrapping text around an image**.

7. CSS Layouts: Columns, Floats and Position Values

Complex **multi-column layouts** can be created with **floats and/or positioning**. A typical dashboard shows:

- a **two-column layout** (heading column + content column),
- a **three-column layout**: the **left column** holds the **navigation**, the **middle** is the main page where the data and visualizations appear, and the **right** is a control area where the user provides input (mostly form elements),
- **nested column layouts within a column** (cards, charts inside the middle column).

Floating Elements

Consider this HTML:

```
<h1>Float example</h1>
<p>A wonderful serenity has taken ...</p>
<figure>
   <img src="images/828.jpg" alt="..." />
   <figcaption>British Museum</figcaption>
</figure>
<p>When, while the lovely valley ...</p>
```

Without float, the page shows paragraph, then the figure (image + caption), then the next paragraph, with a lot of **white space** beside the image. The style sheet (not part of the HTML) styles the figure:

```
figure {
   border: 1pt solid #A8A8A8;
   background-color: #EDEDDD;
   margin: 0;
   padding: 5px;
   width: 150px;
}
```

To place the figure and paragraph **side by side**, add `float` on top of the existing rules:

```
figure {
   ...
   width: 150px;
   float: left;
}
```

The text now wraps around the image, removing the empty space. Note that **a floated block-level element should have a width specified**.

Position Values

Value	Description
`absolute`	Removed from normal flow and positioned **relative to its nearest positioned ancestor**
`fixed`	Fixed at a specific position **in the window**, even when the document is scrolled
`relative`	Moved **relative to where it would be in the normal flow**
`static`	Positioned according to the normal flow. **This is the default**
`sticky`	Positioned per normal flow, then offset relative to its **nearest scrolling ancestor**; lets an item scroll and then **stay fixed** once its scroll position is reached

Together with offsets like `top`, `right`, `left`, you dictate where an element sits. Rather than memorizing definitions, **experience them through hands-on assignments**; they become clear only when you try them.

> **Note for Review** Pro-Tip: Common mix-up: `fixed` is pinned to the **viewport** (it stays while you scroll), while `absolute` is pinned to the nearest **positioned ancestor**. `relative` shifts the element from its *own* normal spot and is also how you turn a parent into the anchor for `absolute` children.

8. Flexbox vs Grid

Take a container with five child `div`s labeled A to E:

```
<div class="container">
  <div>A</div>
  <div>B</div>
  <div>C</div>
  <div>D</div>
  <div>E</div>
</div>
```

```
.container {
   background-color: #4AB2D6;
   width: 600px;
   margin: 50px;
}
.container div { background-color: #80CFEC; ... }
```

Flex version

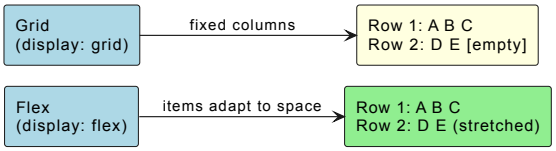
```
.container { display: flex; flex-wrap: wrap; }
.container div { flex: 1 1 150px; }
```

Elements inside a flexbox have a **minimum size of 150px but grow to fit the container**. Result: row 1 holds A, B, C; row 2 holds D and E, and D and E **stretch to use all remaining space**.

Grid version

```
.container {
   display: grid;
   grid-template-columns: repeat(3, 1fr);
}
```

Each row of the grid has **three columns whose size is based on the space available in the container**. A, B, C fill row 1, and D, E fill the first two cells of row 2, leaving the last cell **empty**. The widths are fixed and cells do **not** stretch to occupy leftover space.



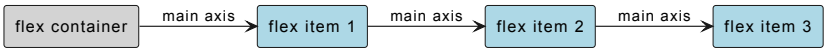
Feature	Flexbox	Grid
Dimension	One-dimensional (a row **or** a column)	Two-dimensional (rows **and** columns)
Item sizing	Items **grow/shrink** to fill space	Fixed track sizes, cells do not adapt
Leftover space	Absorbed by items	May leave empty cells

9. Flexbox in Detail

Container, Items and Axes

A **flex container** holds **flex items**. Items are laid along the **main axis** (horizontal by default), and the **cross axis** runs perpendicular to it. **Both block-level and inline elements become flex items** (a `<span>` inside the container behaves like a `<div>` item).

```
<div class="container">
  <div>flex item 1</div>
  <span>flex item 2</span>
  <div>flex item 3</div>
</div>
```



Use Cases

- A **horizontal navigation bar** — flex can **distribute items evenly along a row**.
- **Data-entry forms** — flex is very helpful for **aligning labels, input controls and buttons**. For instance, "Name" (long) and "Class" (short) fields can be arranged so all of them end up with consistent alignment.

Key Properties

Property	Meaning
`display: flex;`	**Necessary** to use flexbox
`flex-direction: row`	Direction of the main axis; **default is row**
`flex-direction: row-reverse`	Items placed right to left
`flex-direction: column`	Items stacked top to bottom
`flex-direction: column-reverse`	Items stacked bottom to top
`flex-wrap: wrap`	New lines are created if the container cannot hold all items
`flex-wrap: nowrap`	All items forced onto one line

10. Flex and Grid Together

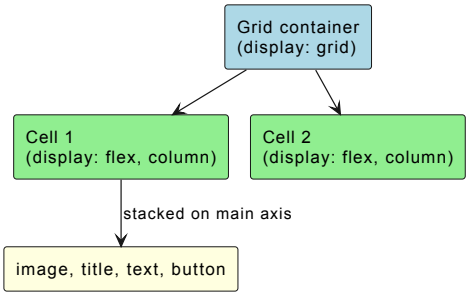
Most of the time you will use **grid and flex together**: grid for the **outer structure**, flex inside each grid cell.

Example: a gallery of artist cards (image, name, caption, button).

```
.container {
  display: grid;
  grid-template-columns: repeat(auto-fill, ... );
}
```

Grid creates the **rows and columns**. To lay out **each individual grid cell**, we use flexbox:

```
.cell {
  display: flex;
  flex-direction: column;
}
.cell img {
  align-self: center;
}
.cell h2, .cell p {
  text-align: center;
}
.cell button {
  margin-top: auto;
  justify-self: flex-end;
  align-self: center;
}
```



This gives you the benefit of both: a clean structured grid outside and adaptive spacing inside each cell.

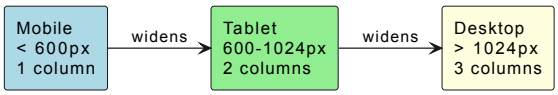
11. Responsive Design

Your page will be rendered in different **environments**: not just different browsers (Internet Explorer, Mozilla, Chrome), but different **resolutions and screen sizes** — laptop, large display, tablet, mobile. A good design adapts to any screen size and gives a similar user experience, which includes how well the user can **interpret** information, how many **scrolls** and **clicks** are needed to browse it.

In the early days, plain HTML pages loaded in mobile browsers gave a painful experience of scrolling and reading. Today this is largely solved.

In a **responsive design**, the page "responds" to changes in browser size, going **beyond simple percentage scaling of widths**: **smaller images are served and navigation elements are replaced** as the window shrinks.

Device	Width	Layout
Mobile	`< 600px`	Single column
Tablet	`600–1024px`	Two columns
Desktop	`> 1024px`	Three columns



The switch is done with **media queries**:

```
@media (min-width: 600px)  { ... }
@media (min-width: 1024px) { ... }
```

Why Not Just Scale Down?

If a page designed for a large screen is merely scaled down to a mobile/tablet screen, everything technically fits, but text and elements **shrink so much that they become practically unusable** — the user experience deteriorates.

The Viewport

The **viewport** is the part of the browser window that displays web content. Using a `meta` tag you tell the browser to scale the layout to the **visible working screen width** (for example it becomes **320px** on a small mobile browser), giving a similar experience across devices:

```
<meta name="viewport" content="width=device-width, initial-scale=1">
```

> **Note for Review** Pro-Tip: Responsive design needs **two ingredients**: a correctly configured **viewport meta tag** and **media queries**. Forgetting the viewport tag makes mobile browsers pretend the screen is wide, and your media queries will not behave as expected.

12. Module Recap: HTML and CSS

- **HTML's skeleton and vocabulary** — doctype, `html`, `head`, `body`, semantic tags, headings, lists, links, tables and forms.
- **Links, tables and forms** are the backbone of interactive pages, each with a defined, accessible structure; they carry the dynamic part, projecting up-to-date information when clients demand it.
- **Forms send data via GET (visible in the URL) or POST (hidden in the body)** — the choice affects **security and caching**, and so performance.
- **CSS pairs a selector with `property: value` rules**; best practice keeps it in an **external style sheet, not inline**.
- **Selectors range from simple** (element, class, id) **to precise** (attribute, contextual/combinator).
- **Conflicting rules are resolved by the cascade**: inheritance, specificity and source order.
- **The box model**: an element's real size = content + padding + border + margin.
- **Flexbox handles one-dimensional alignment; Grid handles two-dimensional layouts.**
- **Responsive design** adapts layout across screen sizes with media queries and a correctly configured viewport.

13. Module 3 Begins: Client-Side Programming with JavaScript

What This Module Covers

- JavaScript background and how to add it to a page (client-side programming)
- Why it is needed
- Making decisions and repeating work: operators, conditionals, loops
- Functions and events (JavaScript is **event-driven**)
- The DOM and forms
- Async basics

If you know C, C++ or Java you will have some advantage, since the basic programming constructs look familiar.

The Big Question

What turns a page that just sits there into one that **responds to every click, keystroke and timer**? Whenever the user interacts with the page — clicking, typing and so on — that interaction is an **event**. On the occurrence of an event, some action must be initiated and **automated**. This is done with JavaScript.

What JavaScript Is

JavaScript is an object-oriented, lightweight, dynamically typed scripting language.

- Why "scripting"? There is no separate compile step producing object code or byte code, as with C/C++ (compiler) or Java (compiler plus Java Virtual Machine). JavaScript is run by an **interpreter**.
- It is **NOT related to Java** other than by name and some syntactic similarities.
- It can be used to **programmatically access** and **dynamically manipulate** any aspect of an HTML document's appearance or content. That is: you can reach different objects through code (automating access, for instance by time or on an event) and change them on demand.
- It can **animate, move, transition, hide and load** content into parts of a page **rather than refresh an entire page from the server** — for example an element moving across the page as it loads.
- It was **introduced by Netscape in their Navigator browser in 1996**, originally called **LiveScript**.
- Mostly used for client-side programming (though it also works server-side; we will see that in a later module).

What It Is Used For

- Making web pages **interactive**
- Inserting **dynamic text** into HTML (e.g. the user's name)
- **Reacting to events** (e.g. page load, user click)
- Getting **information about a user's computer** (e.g. browser type)
- Performing **calculations** on the user's computer (e.g. form validation)

14. Where JavaScript Is Used

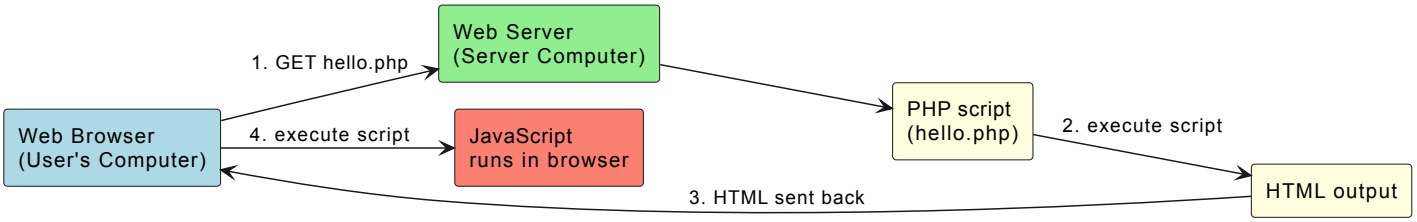
JavaScript is not limited to the browser:

Area	Examples
Non-browser engines, via open-source projects	Google **V8**, Mozilla **SpiderMonkey**, Microsoft **Chakra**
Server-side runtime	**Node.js** (covered in a later server-side module)
Databases	The query language of **MongoDB**
Desktop applications (end-user scripting)	**Adobe Creative Suite**, **OpenOffice**
Embedded / IoT	JavaScript engine in the **Oculus Rift** headset (virtual reality); **Arduino** and **Raspberry Pi** microcontrollers
Native apps	Desktop or mobile applications authored with **React Native** or **Electron**

15. Client-Side Scripting Architecture

The web is a **client–server** architecture: a web browser on the user's computer, a web server on the server computer.

1. **GET**: the browser, using a URL, requests the page `hello.php`.
2. The server **executes the PHP script** (server-side code).
3. The script generates **HTML output**, which is sent back to the browser.
4. The browser renders it and **executes the JavaScript** embedded or referenced in the output.



A natural doubt: PHP can already create dynamic pages, so why not let the server run all the logic and just send simple HTML? That would be a neat design, but client-side scripting brings real advantages.

16. Why Client-Side? Pros and Cons

Advantages

- **Usability** — the page can be modified **without a post-back to the server**. An interactive design driven by the server needs many back-and-forth exchanges, each a **round trip** over GET/POST handshakes with a distant server, adding **latency**. Moving some logic to the client gives a faster, more responsive UI that does not depend heavily on connectivity.
- **Efficiency** — it can make **small, quick changes** to a page **without waiting for the server**; on-demand customization happens locally.
- **Event-driven** — it can respond to user actions like **clicks and keystrokes** immediately.

Disadvantages (On the Con Side)

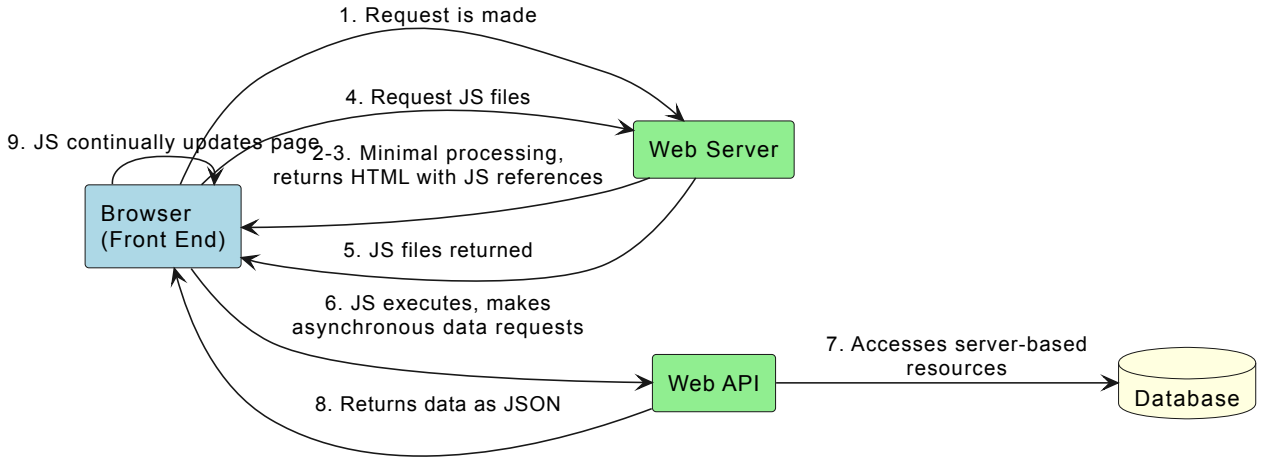
- **No guarantee that the client has JavaScript enabled**, so any required functionality must be **implemented redundantly on the server** — extra overhead.
- **JavaScript-heavy applications can be complicated to debug and maintain**, compared with simple HTML/CSS pages.
- **JavaScript is not fault tolerant.** Browsers cope with invalid HTML or CSS (e.g. showing alternate text if an image is missing, or an error message), but with invalid JavaScript the page **simply stops execution at the invalid line**.
- While JavaScript is **universally supported in all contemporary browsers**, the language and its APIs are **continually expanding**, so **newer features may not be supported in all browsers**. Backward compatibility can be tricky.

Aspect	Benefit	Cost
Speed	No round trips for small changes	—
Availability	—	Client may have JS disabled
Errors	—	Execution halts at the bad line
Maintenance	—	Harder debugging for JS-heavy apps
Compatibility	—	Newer features not on all browsers

---

17. Contemporary JavaScript Coding

In modern applications the work is split between a **front end** (browser) and a **back end** (servers and databases). Notice the trend from a **thicker back end to a thinner back end**, which gives the usability advantages above.



Step by step:

1. A **request is made** from the browser.
2. The server performs **minimal processing** and returns HTML.
3. The HTML arrives with **JavaScript file references**.
4. The browser **requests the JavaScript files** (possibly from another server, or the same location).
5. The **JavaScript files** are returned.
6. JavaScript **executes**, then makes **asynchronous requests** for data from **web APIs** (which may be on the same or different servers).
7. The **web API accesses server-based resources**, such as databases, for the most up-to-date data.
8. Web APIs return the data in **JSON format**.
9. JavaScript **continually updates the page** based on received data and user actions, yielding a dynamic page showing the latest information.

18. Where JavaScript Lives in a Page

There are three places to put JavaScript: **inline**, **embedded**, and **external**.

```
<html lang="en">
<head>
  <title>JavaScript placement possibilities</title>
  <script>
    /* A JavaScript Comment */
    alert("This will appear before any content");
  </script>                                  <!-- Embedded JavaScript -->

  <script src="greeting.js"></script>        <!-- External JavaScript -->
</head>
<body>
<h1>Page Title</h1>

<a href="JavaScript:OpenWindow();">for more info</a>
<input type="button" onClick="alert('Are you sure?');" />
                                             <!-- Inline JavaScript -->
```

Type	How it looks	Verdict
Inline	Written inside an HTML attribute: `href="JavaScript:OpenWindow();"`, or `onClick="alert('Are you sure?');"` (`onClick` is an **event**: on a mouse click, an alert is generated)	**Not recommended** — complicates debugging and error fixing
Embedded	Code between `<script>` and `</script>` in the HTML file, like the alert above	**Not preferable**, same reason
External	A separate `.js` file referenced with `<script src="greeting.js"></script>`	**Recommended and most used** — modular, like an external CSS file; helps segregate scope and makes debugging easier

Notes:

- Embedded JavaScript always carries its **opening and closing `script` tags**, just like HTML markup.
- The external option keeps the JavaScript in a separate file that is fetched whenever needed.
- In the example, the embedded alert appears **before any content** because the script sits in the `head`.

19. Declaring Variables

- `let` declares a variable you **can reassign**; `const` declares one you **cannot** (though an **object or array it points to can still change internally**).
- `var` is the **older way** of declaring variables — **function-scoped rather than block-scoped**, which causes real bugs; **avoid it in new code**.
- JavaScript is **dynamically typed**: you **never declare a type**, and a variable can **hold a different kind of value later in the script**.
- `typeof x` tells you **what type a value currently holds, at runtime**.

```
let count = 5;          // reassignable
count = 6;              // fine

const PI = 3.14159;     // PI = 3; would cause an error

const scores = [10, 20];
scores.push(30);        // allowed: the array's contents may change,
                        // only the binding is fixed

let x = 42;
console.log(typeof x);  // "number"
x = "now a string";
console.log(typeof x);  // "string"
```

> **Note for Review** Pro-Tip: `const` means "this *name* cannot be re-pointed", **not** "this value is frozen". Think of it as a label glued to a box: you cannot move the label to another box, but you can still change what is inside the box.

Coming next: the different data types supported, `for` and `while` loops and other control structures, functions, arrays and the DOM structure.

20. Practice Questions

1. A designer writes `font-family: "Playfair Display", cursive;` for a page used by thousands of people on many different devices. Some users report the heading looks wildly different from device to device. Explain why this happens and how you would redesign the declaration so the page looks close to intended everywhere.

2. A `div` has `width: 300px; padding: 20px; border: 5px solid; margin: 10px;`. Two such `div`s are placed side by side inside a container as inline-blocks. What is the minimum container width needed so that they stay on the same line, and which layers of the box are responsible for the "extra" space you did not explicitly count in the 300px?

3. You are building a product page with a grid of 9 cards. Each card has an image, a title, a description of varying length, and a "Buy" button that must always sit at the bottom of the card, aligned across cards in the same row. Which layout tools (`grid`, `flex`, `float`, `position`) would you assign to which part of this page, and why would using only `float` be a poor choice?

4. A site looks perfect on your laptop. After you shrink the browser, the layout does not change at all, and on a phone the text is tiny and unreadable even though the page "fits". Identify at least two separate things that could be missing, and describe what each would fix.

5. Your page loads a feature using an external script, but a typo on line 12 of the script causes an error. Describe what the user will see, how this differs from a typo in the HTML or CSS, and what the server-side redundancy argument says about whether your page should still work if JavaScript is disabled.